



Claude Code

전사 세미나

2026.04.07

클로드코드 잘 사용하기

AI와 함께하는 새로운 개발 워크플로우

발표자

AI 1팀 김영동 대리

일시

2026년 4월 7일

CLAUDE CODE SEMINAR

목차

- 01 코딩은 어디로 가고 있는가
- 02 왜 Claude Code인가
- 03 Claude Code 기본 사용법
- 04 AI 시대의 개발 방법론
- 05 하네스 & 컨텍스트 엔지니어링
- 06 이렇게 하면 망한다 — 한계와 실패 패턴
- 07 멀티 에이전트 활용
- 08 실전 워크플로우 & 도구 세팅
- 09 비개발자도 할 수 있다
- 10 AI 시대, 우리는 어떻게 해야 하나
- 11 마무리 & Q&A

01

코딩은 어디로 가고 있는가

추상화의 역사 — 1단계

```
01001000 01100101 01101100  
01101100 01101111
```



```
printf("Hello");
```

C 언어

"기계를 직접 다루지 않으면
개발자가 아니다"

추상화의 역사 — 2단계

C

```
ptr = malloc(size);  
// ... use ptr ...  
free(ptr);
```



Java

```
Object obj = new Object();  
// ... use obj ...  
// GC handles cleanup
```

"메모리 직접 관리 안 하면 개발이 아니다"

추상화의 역사 — 3단계

```
print("Hello")
```

~~"장난감 언어, 실제 개발에 못 쓴다"~~

Python → #1 언어

추상화의 역사 — 그리고 지금

"이 함수를 리팩토링하고 테스트를 작성해줘"

~~"이건 진짜 개발이 아니다"~~

매번 틀렸다

25%

YC 2025 겨울 배치 중
코드베이스의 95%를 AI로 생성

Y Combinator

~3개월

Meta 엔지니어가 코드를
직접 타이핑하지 않은 기간

100%

AI가 짠 코드가 사람의 터치 없이
PR에 그대로 올라오는 일상

SECTION 1

마크다운이 코드를 대체하는 현실

이름은 다르지만 역할은 동일 — 마크다운으로 프로젝트 규칙을 AI에게 전달



Claude Code

`CLAUDE.md`



Cursor

`Cursor Rules`



GitHub Copilot

`copilot-instructions.md`



Codex

`AGENTS.md`

개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

건축가

직접 벽돌을 쌓지 않고 설계도를 그린다.
설계도가 정확해야 건물이 올라간다.

영화 감독

직접 연기하지 않고 배우에게 디렉션을 준다.
CLAUDE.md = 대본

그래도 기초는 중요하다

"AI가 더 많이 해줄수록,
기초 지식을 가진 사람의 경쟁력이 역설적으로 올라간다"

변하는 것

직접 코드를 타이핑하는 비중

데모까지는 쉽지만, 실서비스로 만드는 건 전혀 다른 레벨

변하지 않는 것

시스템을 이해하는 능력

보안, 동시접속, DB 스키마, 캐싱 — 기본기 없이는 질문조차 불가

02

왜 Claude Code인가

AI 코딩 도구의 진화 3단계

1단계

자동완성

IDE가 다음 줄을 예측
반복 패턴에 탁월

한계: 현재 파일로 컨텍스트 제한

2단계

대화형 채팅

코드 토론, 버그 설명
스니펫 붙여넣기

한계: 다중 파일 조율이 수동

3단계

에이전틱 코딩

목표를 분해하고 독립 실행
전체 코드베이스 이해

← 지금 여기

SECTION 2

개발 패러다임 전환

기존 개발

코드 작성



테스트 실행



오류 읽기



수정



반복...

에이전틱 개발

목표 정의



변경 검토



구현 승인

에이전틱 코딩의 실제 성과

2주

CTO 추정 4~8개월 프로젝트를
에이전틱 코딩으로 완료

Augment Code + Vertex AI

1~2일

신규 개발자 온보딩
기존 수주~수개월에서 단축

첫날부터 의미 있는 기여 가능

복합 효과

불가능해 보이던 작업 실현
기술 부채 점진적 해결
즉각적 사용자 피드백 대응

SECTION 2

AI 코딩 도구 비교

도구	형태	특징
GitHub Copilot	IDE 플러그인	코드 자동완성 중심, 가장 대중적
Cursor	IDE	AI 네이티브 에디터, 에이전틱 기능 내장
Windsurf	IDE	에이전틱 IDE, Cascade 에이전트
Codex (OpenAI)	터미널 CLI	에이전틱, 클라우드 샌드박스 실행
Claude Code	터미널 CLI	에이전틱, 로컬 실행, 확장 레이어 구조

SECTION 2

CLI 기반 — IDE에 종속되지 않음

터미널이면 어디서든 실행. 스크립트화, 자동화, CI/CD 통합에 유리.



터미널 CLI

핵심 인터페이스



VS Code

확장 프로그램



JetBrains

확장 프로그램



데스크톱 앱

Mac / Windows



웹

claude.ai/code



Chrome 확장

브라우저 통합

에이전트 루프

목표만 주면 작업 완료까지 알아서 반복. 언제든지 중단하여 방향 수정 가능.

1

컨텍스트 수집

파일 검색, 코드 구조 파악
설정 파일 읽기, 의존성 매핑



2

작업 수행

파일 편집, 셀 명령 실행
새 파일 생성



3

결과 검증

테스트 실행, 빌드 확인
오류 분석 → 1로 복귀

SECTION 2

내장 도구 — 5가지 범주, 20개+

파일 작업

파일 읽기, 코드 편집, 새 파일 생성, 이름 변경 및 재구성

검색

패턴으로 파일 찾기, 정규식으로 콘텐츠 검색, 코드베이스 탐색

실행

셸 명령 실행, 서버 시작, 테스트 실행, git 사용

웹

웹 검색, 문서 가져오기, 오류 메시지 조회

코드 인텔리전스

타입 오류/경고 확인, 정의로 이동, 참조 찾기

확장 레이어 구조

처음에는 CLAUDE.md만으로 시작. 필요에 따라 하나씩 추가한다.

CLAUDE.md	모든 세션에서 로드되는 컨텍스트	여기서 시작
Skills	재사용 가능한 워크플로우	반복 작업이 생길 때
MCP	외부 서비스 연결 (100개+)	Notion, GitHub, Slack 등
Subagents	격리된 컨텍스트, 요약 반환	복잡한 조사/병렬 작업
Agent Teams	여러 독립 세션이 협업	대규모 병렬 작업
Hooks	이벤트 기반 자동화 스크립트	링크, 테스트 자동화
Plugins	기능 패키징 및 배포	팀/커뮤니티 공유

Subagent vs Agent Team

둘 다 작업을 위임하는 방식. 규모와 독립성에 따라 선택.

Subagent

구조

메인이 생성한 격리된 워커

컨텍스트

자체 윈도우, 요약만 반환

통신

주 에이전트에게만 보고

많은 파일 읽기, 병렬 조사

Agent Team

구조

여러 독립 세션이 협업

컨텍스트

완전히 독립, 각자 별도 윈도우

통신

팀원끼리 직접 메시지

보안/성능/테스트 동시 검토

빠르고 효율적

Sonnet

대부분의 코딩 작업에 적합

빠른 응답 속도

일상적인 개발 워크플로우

기본 모델

깊고 정교한

Opus

복잡한 아키텍처 결정

깊은 분석이 필요한 경우

대규모 리팩토링

/model 로 즉시 전환

권한 모드와 안전장치

권한 모드 — Shift+Tab으로 전환

default

각 도구 처음 사용 시 권한 요청

acceptEdits

파일 편집 자동, 명령은 요청

plan

읽기 전용 — 분석만 가능

auto

배경 안전 검사로 자동 승인

안전장치

체크포인트

모든 파일 편집은 되돌릴 수 있음

Esc 두 번으로 복원

도구별 승인

읽기 전용 → 자동 허용

Bash/파일 수정 → 승인 필요

세밀한 규칙

/permissions로 Allow/Ask/Deny 설정

deny가 항상 최우선

왜 생태계가 중요한가

도구의 기능은 시간이 지나면 경쟁사도 따라잡는다.
하지만 커뮤니티가 만든 플러그인, 스킬, 노하우, 교육 자료는
쉽게 복제되지 않는다.

문제가 생겼을 때 검색하면 답이 나오는 환경
= 실무 도입의 핵심 조건

03

AI 시대의 개발 방법론

왜 방법론이 필요한가

에이전틱 코딩이 등장하면서, 병목이 바뀌었다.

이전의 병목

코드를 작성하는 것

지금의 병목

AI에게 **무엇을** 시킬지 정하는 것
+ 정말 원하는 대로 했는지 **확인**하는 것

타임라인

2025.02

Karpathy "Vibe Coding" 명명

AI 코딩의 대중화, 문화적 전환점

2025 중반

AI 생성 코드의 품질/보안 문제 부각

구조화된 방법론의 필요성 대두

2025 하반기

SDD, BMAD 등 프레임워크 등장

Vibe Coding의 한계에 대한 업계 대응

2026 초

TDD + AI, Context Engineering 부상

성숙한 AI 개발 워크플로우의 정립

SECTION 3

구조화 수준 스펙트럼

왼쪽으로 갈수록 자유롭고, 오른쪽으로 갈수록 엄격하다.

VDD

바이브 코딩

탐색적

PDD

프롬프트 주도

의도

ADD

에이전트 주도

협업

TDD+AI

테스트 주도

검증

SDD

스펙 주도

계획

BMAD

애자일 AI

조직

낮은 구조화 ← → 높은 구조화

VDD — 바이브 코딩

"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists."

— Andrej Karpathy, 2025.02

적합한 경우

프로토타이핑, MVP

주말 사이드 프로젝트

아이디어 탐색

Collins Dictionary 2025 올해의 단어
검색량 6,700% 급증

현실적 문제

AI 생성 코드의 ~45%에 보안 결함

"3개월 블랙박스" — 아무도 설명 못 함

"시작하는 데는 괜찮지만 유지보수에는 끔찍" — Linus Torvalds

PDD — 프롬프트 주도 개발

프롬프트를 위시리스트가 아닌, **검증 가능한 명세(contract)**로 취급한다.

좋은 PDD 프롬프트

```
"사용자 인증 시스템을 구현하라.  
- 입력: email(RFC 5322),  
      password(최소 8자, 대소문자+숫자+특수)  
- 출력: JWT 토큰(24h 만료)  
      또는 에러 코드(401/429)  
- 제약: 실패 5회 → 15분 잠금  
응답 p99 < 200ms"
```

나쁜 PDD 프롬프트

```
"로그인 기능 만들어줘"
```

PDD 원칙

"완료"를 정의할 수 있을 때만 코드 작성
프롬프트가 측정 가능해질 때까지 반복
테스트는 심판이지 장식이 아니다

ADD — 에이전트 주도 개발

"어떻게 구현할까?" → "에이전트가 올바르게 구현하도록 어떻게 안내할까?"

에이전트가 하는 것

코드 검색, 읽기, 작성
셸 명령 실행, 테스트
패키지 설치, 의존성 관리
문서 생성

인간이 소유하는 것

전략적 의사결정 (무엇을 만들지)
아키텍처 선택
보안 검증
최종 품질 검증

인간이 테스트를 쓰고, AI가 테스트를 통과하는 코드를 구현한다.

1. [인간]

실패 테스트(Red) 작성

2. [AI]

테스트를 통과하는 최소 코드(Green)

3. [인간]

통과 코드를 감사

4. [AI]

통과 상태 유지하며 리팩토링

핵심 원칙

테스트가 우리(cage)다

인간이 테스트, AI가 구현

한 번에 하나의 테스트

테스트를 수정하여 통과시키지 않는다

40~80%

버그 감소 — 2025 DORA 보고서

SECTION 3

SDD — 스펙 주도 개발

코드가 아닌 **스펙이 소스 오브 트루스**. Vibe Coding 이후 업계의 궤도 수정.

워크플로우

[인간] 요구사항을 구조화된 스펙으로 작성

[AI] 스펙을 읽고 구현 계획 제안

[인간] 계획 검토 및 승인

[AI] 스펙에 따라 코드 생성

[자동화] 스펙 준수 여부 검증

스펙 예시

Feature: User Auth

Requirements

- OAuth 2.0 via Google, GitHub
- JWT with 24-hour expiry
- Rate limit: 5 fails → 15min lock

Constraints

- PostgreSQL database
- Response < 200ms at p99

Acceptance Criteria

- [x] Google OAuth sign-in
- [x] Invalid tokens → 401
- [x] Rate limit after 5 failures

SECTION 3

방법론 비교

	Vibe	PDD	ADD	TDD+AI	SDD
인간의 역할	묘사자	프롬프트 작성자	아키텍트	테스트 작성자	스펙 작성자
검증 방식	감각적	출력 비교	CI/CD	자동 테스트	스펙 검증
진입 장벽	매우 낮음	낮음	높음	중간	중간~높음
프로덕션	낮음	중간	높음	높음	높음
유지보수	매우 낮음	중간	높음	높음	높음

결론

Spec + TDD

Spec

"무엇을" 정의한다
의도를 명확히 전달

+

TDD

"정말 그렇게 됐는지" 검증
결과를 자동으로 확인

스펙 없이 TDD만 하면 — 나무는 보지만 숲을 못 본다

TDD 없이 스펙만 쓰면 — 검증할 수단이 없다

그런데 이걸 매번 수동으로 할 것인가?

→ CLAUDE.md에 **스펙 템플릿**을 넣고

→ Skills로 **TDD 워크플로우**를 패키징하고

→ Hooks로 **테스트 자동 실행**을 걸면

이것이 바로 **하네스 엔지니어링**이다

05

하네스 & 컨텍스트 엔지니어링

06

이렇게 하면 망한다

한계와 실패 패턴



07

멀티 에이전트 활용

08

실전 워크플로우 & 도구 세팅

09

비개발자도 할 수 있다

10

AI 시대,
우리는 어떻게 해야 하나

11

마무리 & Q&A



Claude Code

감사합니다

질문이 있으시면 편하게 말씀해주세요

